

BeeWarm

*Système de chauffage d'appoint pour ruches apicoles
alimenté par énergie solaire photovoltaïque*



Mémoire de Projet

ELSE4 FISA — Polytech Nice Sophia

Réalisé par :

Anas ESSALIMI · Hussein ELSOUDA · Eya ZAOUN · Valentin GIROD-ROUX

Sous la supervision de : M. Peter Christian

Année universitaire 2025-2026



Table des matières



1. Résumé / Introduction	3
2. Contexte et objectifs du projet	4
3. Architecture globale du système	5
4. Liste du matériel	6
5. Conception électronique (PCB)	7
6. Développement logiciel (Arduino / ESP32)	9
7. Application mobile BeeWarm (MIT App Inventor)	11
8. Conception mécanique (modélisation 3D & fabrication)	12
9. Assemblage et tests	13
10. Résultats et perspectives	14
11. Annexes / Bibliographie	15

1. Résumé / Introduction

1.1 Objectif du projet

Le projet BeeWarm s'inscrit dans le cadre du cours de Projet Électronique de quatrième année de la formation Ingénierie Electronique et Systèmes Embarqués (ELSE4 FISA) de Polytech Nice Sophia-Antipolis, sous la supervision de M. Peter Christian.

Son objectif est de concevoir et réaliser un système de chauffage d'appoint autonome pour les ruches d'abeilles situées sur le campus. La finalité principale est d'aider les colonies à maintenir la température interne de la ruche autour de 33 °C, valeur critique pour le développement du couvain, en s'appuyant uniquement sur l'énergie solaire photovoltaïque.

Ce projet est conçu pour être une base de travail pérenne : les promotions suivantes pourront l'améliorer et y ajouter de nouvelles fonctionnalités.

1.2 Ce que le système fait

Le système récupère l'énergie produite par un panneau solaire photovoltaïque pour alimenter un microcontrôleur ESP32. Ce microcontrôleur est chargé de :

- relever en temps réel la température intérieure de la ruche et la température de la végétaline (corps de chauffe) grâce à des capteurs DS18B20 ;
- mesurer la puissance délivrée par le panneau solaire via un capteur INA219 ;
- adapter automatiquement la puissance de chauffe des résistances (par modulation de largeur d'impulsion – PWM) selon un algorithme de type Perturb & Observe (MPPT) ;
- communiquer en Bluetooth avec une application mobile dédiée (BeeWarm) pour afficher les données en temps réel et permettre à l'utilisateur de modifier la consigne de température.

1.3 Ce que le document contient

Ce mémoire présente l'ensemble du travail effectué par le groupe tout au long des séances de projet, depuis la reprise du travail de l'ancien groupe jusqu'à la phase d'assemblage final. Il décrit successivement l'architecture du système, la conception du PCB, le développement logiciel, l'application mobile, la partie mécanique, les essais réalisés et les perspectives d'amélioration.

2. Contexte et objectifs du projet



2.1 Contexte apicole

Les abeilles maintiennent naturellement la température de leur couvain entre 32 °C et 35 °C. En période hivernale ou lors des journées froides du printemps et de l'automne dans les Alpes-Maritimes, cette régulation peut s'avérer difficile et consommer une quantité importante des réserves énergétiques de la colonie. Un apport de chaleur extérieur maîtrisé peut donc significativement améliorer la survie de l'essaim et sa productivité.

M. Peter, qui supervise le projet, possède plusieurs ruches sur le campus de Polytech Nice Sophia-Antipolis. C'est dans ce contexte que ce projet a été lancé il y a un an par un premier groupe d'étudiants (Simon Bressot, Melilla Caillot, Laura Boivin), puis repris et poursuivi par notre groupe.

2.2 Héritage du projet précédent

L'ancien groupe a posé les bases du système : conception du schéma électronique sur Altium Designer, caractérisation du panneau solaire, mise en place d'une première version du code Arduino avec MPPT et Wi-Fi, et réalisation d'un PCB. Notre groupe a repris ce travail, avec pour objectif de le rendre pleinement fonctionnel :

- adaptation du PCB sous Eagle (reprise du schéma, routage, fabrication d'une nouvelle version) ;
- passage de la communication Wi-Fi au Bluetooth pour plus de simplicité d'usage ;
- développement complet de l'application mobile MIT App Inventor ;
- conception et impression 3D du système de suivi solaire motorisé ;
- assemblage, tests et mise en service.

Points clés du contexte

- Ruches situées sur le campus de Polytech Nice Sophia-Antipolis (Alpes-Maritimes)
- Température cible du couvain : ~33 °C
- Source d'énergie : panneau solaire photovoltaïque (autonomie totale)
- Interface utilisateur : application mobile Android via Bluetooth

3. Architecture globale du système

Le système BeeWarm se décompose en plusieurs blocs fonctionnels étroitement couplés :

Bloc fonctionnel	Rôle	Composants principaux
Captation d'énergie	Convertir l'énergie solaire en électricité	Panneau PV, câblage
Suivi solaire	Orienter le panneau vers le soleil	Moteur pas à pas, photo-résistances, ESP32 externe, engrenages 3D
Mesure de puissance	Surveiller V, I, P du panneau	Capteur INA219 (I ² C)
Régulation MPPT	Maximiser l'énergie extraite	Algorithme P&O sur ESP32, PWM (LEDC)
Chauffage	Élever la température de la végétaline	Résistances chauffantes, transistors PMOS, végétaline
Mesure de température	Surveiller T ruche et T végétaline	Capteurs DS18B20 (bus 1-Wire), GPIO 14
Communication	Échanger données avec l'app mobile	Module Bluetooth ESP32 (BeeWarmESP32)
Application mobile	Interface utilisateur	MIT App Inventor, Android
Alimentation	Distribuer l'énergie aux circuits	Régulateur de tension, condensateurs de découplage

3.1 Schéma fonctionnel simplifié

Le flux d'information et d'énergie suit le chemin suivant : le panneau solaire génère une tension qui alimente la PCB via un régulateur de tension. L'INA219 mesure en permanence la puissance disponible sur le bus. L'ESP32 ajuste le rapport cyclique du PWM pilotant les transistors PMOS pour maximiser la puissance transférée aux résistances de chauffe (MPPT). En parallèle, les capteurs DS18B20 remontent les températures ; si la ruche descend en dessous de la consigne (33 °C par défaut, modifiable via l'app), le chauffage s'active.

Les données (température ruche, température végétaline) sont envoyées toutes les 3 secondes par Bluetooth à l'application mobile, sous la forme : DATA;T_ruche°C;T_végétaline°C.

4. Liste du matériel

4.1 Logiciels utilisés

- Arduino IDE 2.x — développement et téléversement du code ESP32
- Eagle (Autodesk) — conception du schéma électronique et routage PCB
- OnShape — modélisation 3D des pièces mécaniques
- MIT App Inventor — développement de l'application mobile Android
- Cameo (Dassault Systèmes) — modélisation système (diagrammes d'exigences, cas d'usage)
- Canva — conception du logo et de l'identité visuelle
- GitHub — gestion du code source

4.2 Composants électroniques principaux (BOM)

Référence	Désignation	Quantité	Rôle
ESP32	Microcontrôleur ESP32 WROOM	1	Cerveau du système, BT, PWM, I ² C
INA219	Capteur de courant/tension I ² C	1	Mesure P, V, I panneau solaire
DS18B20	Capteur température numérique 1-Wire	5	4 végétaline + 1 ruche (déporté)
PMOS	Transistors P-channel MOSFET	2	Commutation résistances de chauffe
NMOS	Transistors N-channel MOSFET	2	Commande des PMOS depuis l'ESP32
AP331AWG-7	Comparateur à drain ouvert	1	Protection / signalisation
Résistances chauffantes	Résistances de puissance	N/A	Chauffe de la végétaline
Régulateur de tension	Régulateur 3.3V/5V	1	Alimentation stable des circuits
Condensateurs	Découplage et filtrage	Plusieurs	Stabilisation alimentation
Résistances pull-up/down	RPU, RPD	6+	Niveaux logiques stables
Rshunt	Résistance shunt série	1	Mesure de courant INA219
Photo-résistances	LDR	2	Détection position soleil
Moteur pas à pas	Moteur suivi solaire	1	Orientation panneau solaire

4.3 Matériel de laboratoire

- Oscilloscope numérique
- Alimentation de laboratoire réglable
- Multimètre
- Fer à braser, four de brasage (préchargé)
- Imprimantes 3D Prusa MK4 (PETG) — FabLab Polytech
- Ponceuse à bande Metabo BS 175
- Kit panneau solaire (panneau, solarimètre, rhéostat, autotransformateur)
- Carte FTDI (communication série FTDI232 entre ESP32 et PC)

5. Conception électronique (PCB)

La première étape a consisté à sécuriser la base schématique sous Eagle. Les composants importés depuis SmartEDA et SnapEDA ont été vérifiés afin de confirmer la présence d'un symbole, d'une empreinte PCB et d'une correspondance correcte entre le boîtier choisi et la référence réelle. Cette vérification était essentielle, car un composant possédant uniquement un symbole schématique, sans footprint fiable, aurait pu rendre le routage ou la fabrication impossible. Le schéma a aussi été comparé à l'ancienne version réalisée sous Altium pour éviter les écarts avec les choix techniques précédents.

5.1 Schématique sous Eagle

La conception du schéma électronique a été réalisée sous Eagle. L'architecture adoptée s'articule autour de l'ESP32, qui centralise la commande de l'ensemble du système. Le schéma comprend les blocs suivants :

- Alimentation : régulateur de tension fournissant 3,3 V et 5 V aux différents circuits à partir de la tension du panneau solaire (jusqu'à 25 V) ;
- Mesure de puissance : l'INA219 est connecté en I²C (SDA sur GPIO 21, SCL sur GPIO 22). Une résistance shunt (Rshunt) placée en série permet la mesure de courant ;
- Commande de chauffe : deux NMOS commandent deux PMOS (via résistances pull-up RPU1 et RPU2). Les résistances pull-down RPD1 et RPD2 maintiennent les grilles des NMOS à la masse en l'absence de signal, évitant toute activation accidentelle ;
- Capteurs de température : quatre DS18B20 partagent un bus 1-Wire (GPIO 14) avec résistance pull-up RPU4 ; un cinquième capteur est déporté à l'extérieur de la carte pour mesurer la température à l'intérieur de la ruche ;
- Comparateur AP331AWG-7 : fonctionne en drain ouvert, avec résistance pull-up RPU3 sur la broche Enable ;
- Pont diviseur R3/R4 : abaisse la tension du panneau à un niveau mesurable par le comparateur.

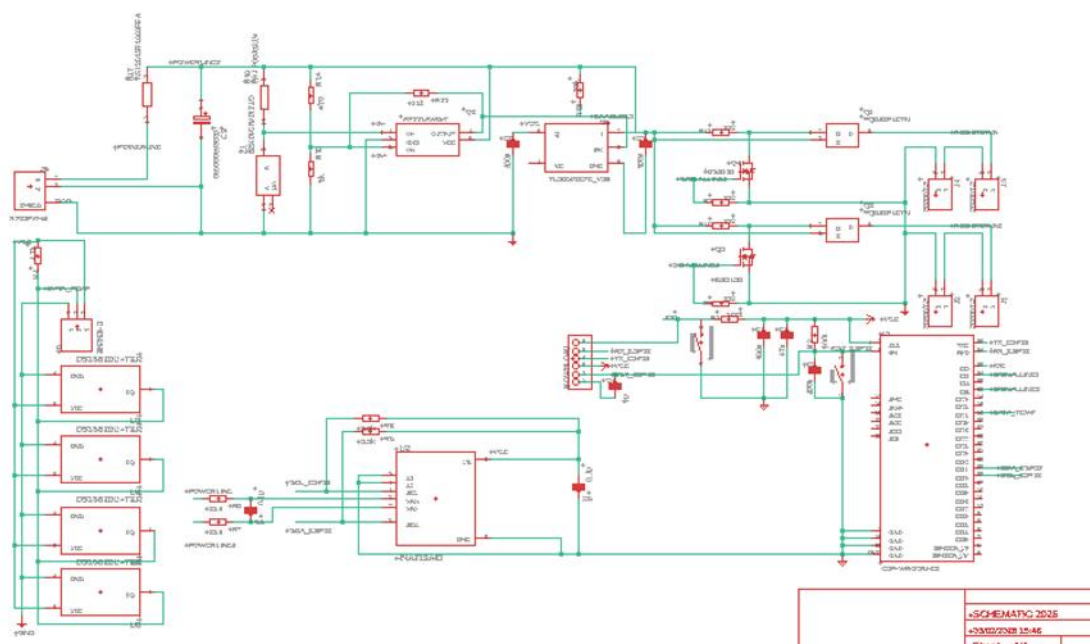


Figure 1 — Vue globale du schéma électronique sous Eagle après reprise et vérification des composants.

5.2 Passage du schématique au layout

Après la validation du schéma, le travail s'est poursuivi par le placement initial des composants sur la board. Les composants ont été organisés par blocs fonctionnels afin de garder une carte lisible : alimentation, commande, mesure, connecteurs et zones liées au chauffage. Les condensateurs de découplage ont été rapprochés des composants qu'ils stabilisent, et les connecteurs ont été maintenus en périphérie pour faciliter les raccordements externes.

Cette phase de placement a aussi nécessité plusieurs rotations de composants. L'objectif était de réduire le croisement des airwires et de préparer un routage plus simple. Avant même de tracer les pistes, cette organisation conditionne fortement la qualité finale du PCB : une implantation mal pensée conduit à des pistes plus longues, davantage de vias et un routage moins robuste.

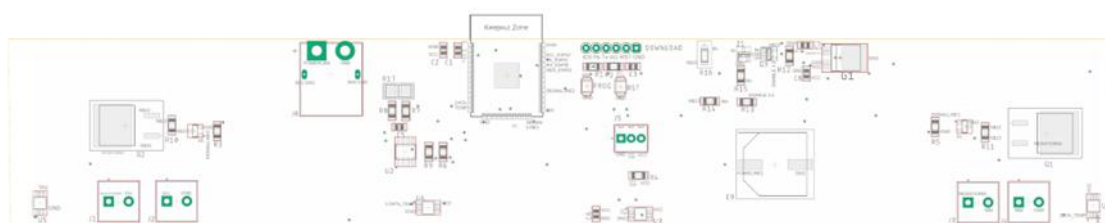


Figure 2 — Première implantation des composants sur la board après passage du schématique au layout.

5.3 Routage de la carte et choix techniques

Le routage a consisté à transformer progressivement les airwires en pistes physiques sur les couches Top et Bottom du PCB. Un premier essai avec l'autorouteur a permis d'identifier les zones complexes, mais le résultat final a été repris manuellement pour obtenir un tracé plus propre et mieux contrôlé.

Plusieurs règles ont été appliquées pendant cette étape : limiter les détours, éviter les croisements problématiques, garder des espacements suffisants entre pistes et utiliser les vias uniquement lorsque le changement de couche était nécessaire. Les pistes d'alimentation, notamment VCC, ont été dessinées plus larges que les signaux classiques afin de mieux supporter le courant, de limiter les chutes de tension et de renforcer la robustesse de la carte.

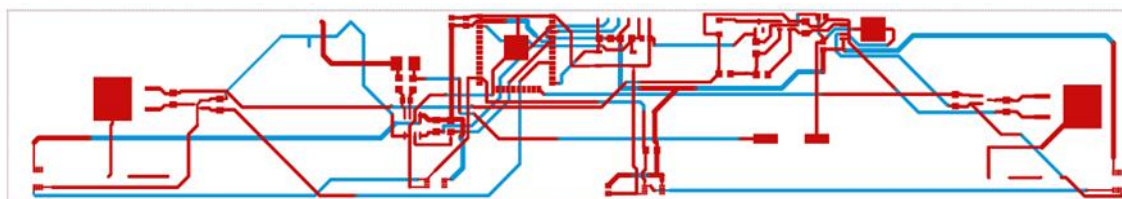


Figure 3 — Vue d'ensemble du routage de la carte avec utilisation des deux couches du PCB.

5.4 Vérification, correction des erreurs et génération des fichiers

Une partie importante du travail a été la correction des erreurs restantes après le routage. Même lorsque la carte semblait correcte visuellement, plusieurs défauts subsistaient : pistes proches mais non reliées électriquement, jonctions incomplètes, espacements insuffisants ou

incohérences avec les règles de fabrication. Environ une soixantaine de petites erreurs ont donc été reprises une par une.

La vérification sous Eagle a permis de valider la cohérence entre le schéma et la board avant l'envoi en fabrication. Une fois les erreurs bloquantes corrigées, les fichiers Gerber ont été exportés puis regroupés dans une archive ZIP. Cette étape a marqué la fin de la phase de conception logicielle du PCB et le passage vers la fabrication physique.

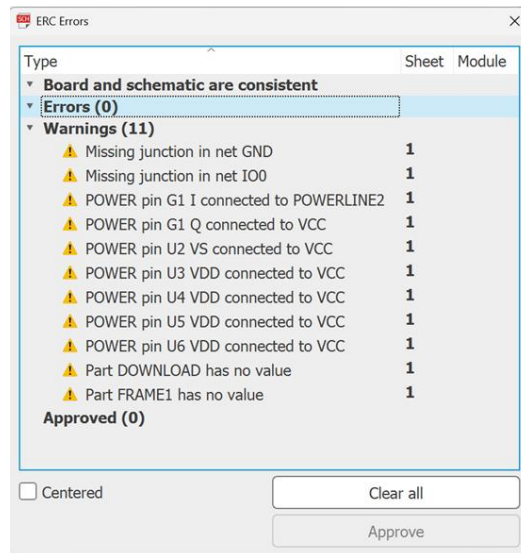


Figure 4 — Fenêtre de vérification Eagle montrant la cohérence entre schéma et board après correction.

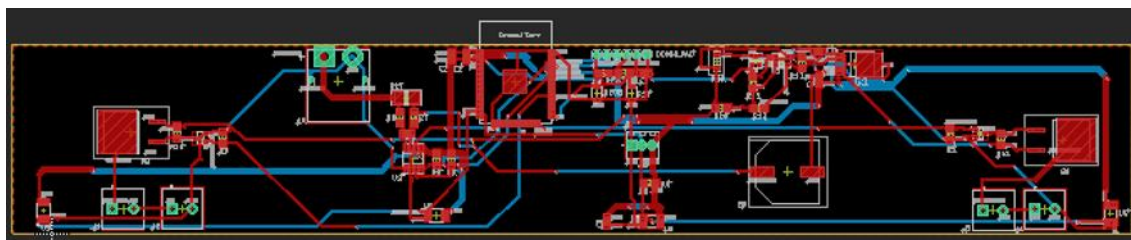


Figure 5 — Carte finalisée après corrections, prête pour l'export des fichiers Gerber.

5.5 Assemblage de la PCB et brasage CMS

Après réception de la carte, la phase d'assemblage a commencé par le dépôt de pâte à braser à l'aide d'un pochoir métallique. Le pochoir devait être correctement aligné afin de déposer la pâte uniquement sur les pastilles prévues et d'éviter les ponts d'étain entre pads proches.

Les composants CMS ont ensuite été placés à la pince, en commençant par les éléments simples comme les résistances et condensateurs, puis en poursuivant avec les composants principaux : ESP32, composants de puissance, éléments de conditionnement et capteur de mesure. Le passage au four de refusion a permis de faire fondre la pâte à braser et de fixer les composants sur les pads..

Cette phase a également mis en évidence une erreur de référence sur le composant de mesure de courant de type INA. Une référence différente de celle utilisée l'année précédente avait été sélectionnée ; son brochage n'était donc pas compatible avec l'empreinte prévue sur la PCB. Il a fallu identifier cette incompatibilité, prévoir une référence adaptée et attendre le bon composant pour finaliser cette partie

Lors des premiers tests sous alimentation, un problème d'alimentation de la Powerline a été détecté (aucun courant tiré par la carte). Après analyse du schéma, le défaut a été corrigé en soudant un câble directement sur la PCB, permettant de rétablir le bon fonctionnement.

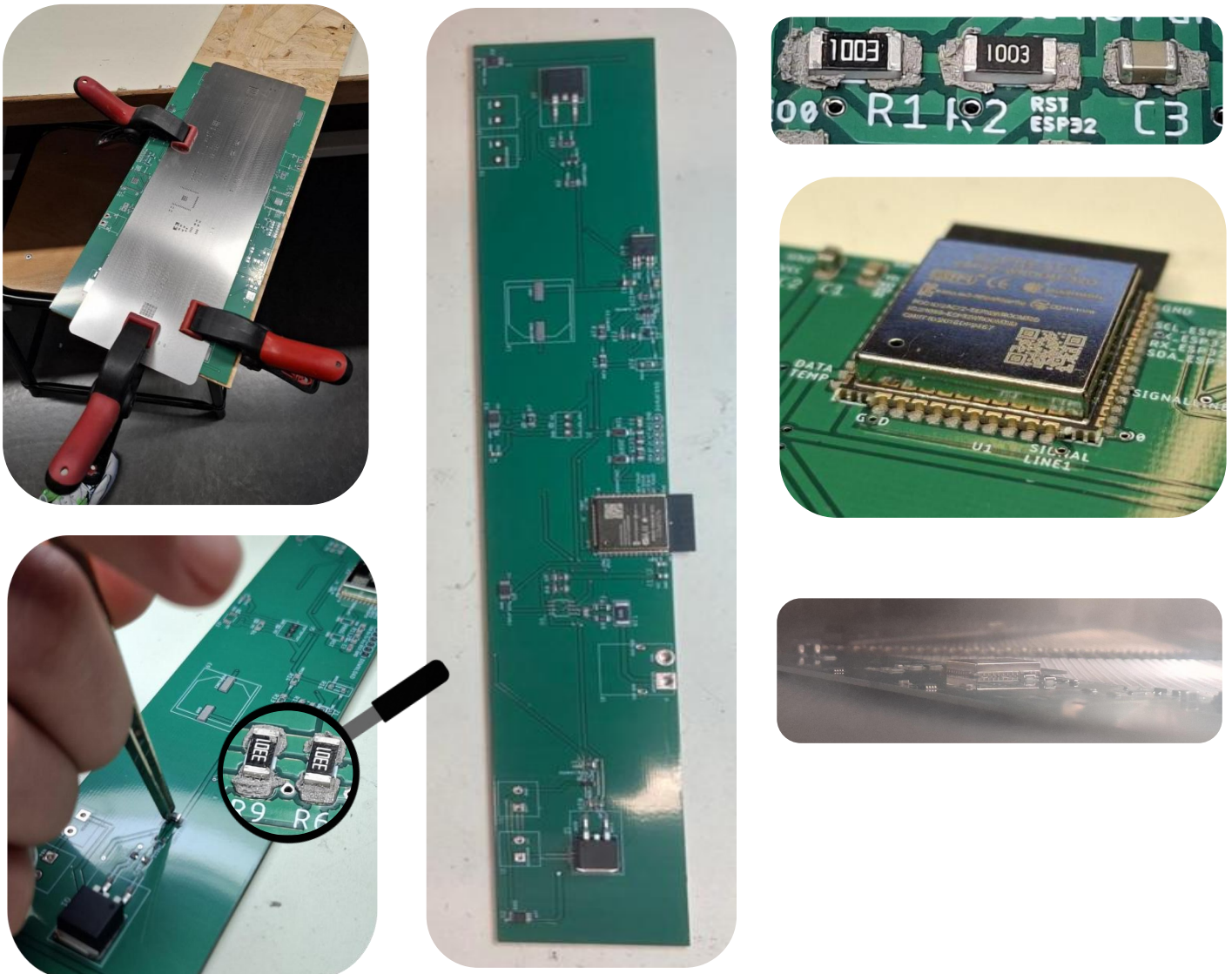


Figure 8 — Premières carte assemblée après dépôt de pâte, placement des composants et brasage.

Cette partie du projet a donc transformé l'architecture électronique de BeeWarm en une carte réelle, utilisable pour les tests d'alimentation, de mesure, de chauffage et de communication. .

6. Développement logiciel (Arduino / ESP32)



6.1 Structure générale du programme

Le code Arduino tourne sur l'ESP32 embarqué dans la PCB (nom Bluetooth : BeeWarmESP32). Il s'exécute en boucle principale (loop) à un rythme de 3 secondes et accomplit les tâches suivantes à chaque cycle :

1. Réception d'une éventuelle commande de consigne via Bluetooth (format : "TEMP=XX") ;
2. Acquisition des températures sur le bus 1-Wire (DS18B20) ;
3. Envoi des données Bluetooth sous la forme : DATA;T_ruche°C;T_végétaline°C ;
4. Lecture des mesures INA219 (tension bus, tension shunt, courant, puissance) ;
5. Application de la logique d'hystérésis thermique (± 1 °C autour de la consigne) ;
6. Mise à jour du rapport cyclique alpha par l'algorithme MPPT P&O ;
7. Écriture du rapport cyclique sur les deux canaux PWM (SIGNAL1 GPIO 2, SIGNAL2 GPIO 5).

6.2 Gestion des capteurs de température DS18B20

Les capteurs DS18B20 utilisent le protocole 1-Wire, géré par les bibliothèques OneWire et DallasTemperature. La bibliothèque détecte automatiquement tous les capteurs présents sur le bus et les identifie par leur adresse matérielle unique sur 8 octets.

Le capteur dédié à la mesure de température dans la ruche est identifié par comparaison d'adresse (memcmp) à l'adresse prédéfinie capteurRuche. Les capteurs restants mesurent la température de la végétaline ; leur valeur moyenne est calculée et transmise.

6.3 Mesure de puissance solaire — INA219

L'INA219 est initialisé sur le bus I²C (SDA GPIO 21, SCL GPIO 22) avec la calibration 32 V / 2 A (setCalibration_32V_2A). À chaque cycle, le programme lit :

- la tension de bus (getBusVoltage_V) ;
- la tension shunt en mV (getShuntVoltage_mV) ;
- le courant en mA (getCurrent_mA) ;
- la puissance en mW (getPower_mW).

6.4 Algorithme MPPT — Perturb & Observe (P&O)

L'algorithme Perturb & Observe adapte le rapport cyclique alpha (compris entre 0,05 et 0,95) du signal PWM pilotant les résistances de chauffe, afin de maintenir le panneau solaire au voisinage de son point de puissance maximale (MPP).

À chaque itération, les variations ΔP et ΔV sont calculées par rapport aux valeurs précédentes. La direction de perturbation est déterminée par le signe des deux variations :

- $\Delta P > 0$ et $\Delta V > 0 \rightarrow$ direction = -1 (réduire alpha pour revenir vers le MPP) ;
- $\Delta P > 0$ et $\Delta V < 0 \rightarrow$ direction = +1 ;
- $\Delta P < 0$ et $\Delta V > 0 \rightarrow$ direction = +1 ;
- $\Delta P < 0$ et $\Delta V < 0 \rightarrow$ direction = -1.

La mise à jour n'est effectuée que si les variations dépassent des seuils minimaux ($|\Delta V| > 10$ mV et $|\Delta P| > 50$ mW), afin d'éviter les oscillations parasites autour du MPP. Le rapport cyclique résultant est converti en valeur entière (duty = alpha $\times$ 255) et appliqué via ledcWrite aux deux sorties PWM.

6.5 Régulation thermique par hystérésis

Une bande d'hystérésis de ± 1 °C autour de la consigne (33 °C par défaut) est implémentée pour éviter les commutations trop fréquentes des résistances de chauffe. Le chauffage s'active uniquement si $T_{\text{ruche}} < (\text{consigne} - 1 \text{ °C})$ et s'éteint si $T_{\text{ruche}} > (\text{consigne} + 1 \text{ °C})$. Lorsque le chauffage est actif, le duty cycle issu du MPPT est appliqué ; sinon, les deux canaux sont mis à zéro.



6.6 Communication Bluetooth

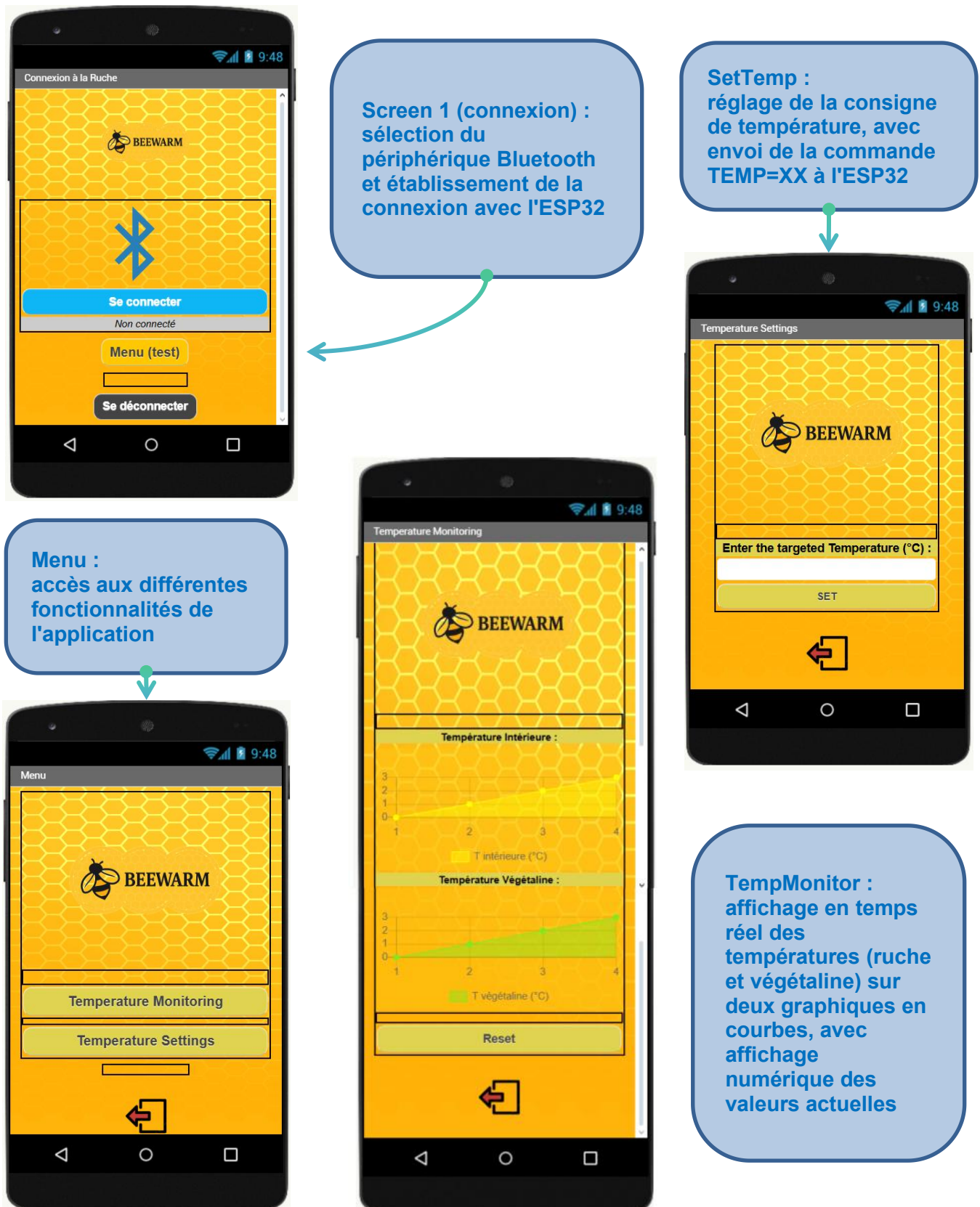
La communication utilise la bibliothèque BluetoothSerial native de l'ESP32. L'appareil se déclare sous le nom BeeWarmESP32 lors de l'initialisation. Il envoie les données de température toutes les 3 secondes à l'application mobile connectée, et écoute en permanence les messages de consigne entrants. La consigne reçue est validée (valeur entière strictement positive et inférieure à 60 °C) avant d'être prise en compte.

7. Application mobile BeeWarm (MIT App Inventor)



7.1 Architecture de l'application

L'application BeeWarm a été développée avec MIT App Inventor. Elle est organisée en quatre écrans indépendants :



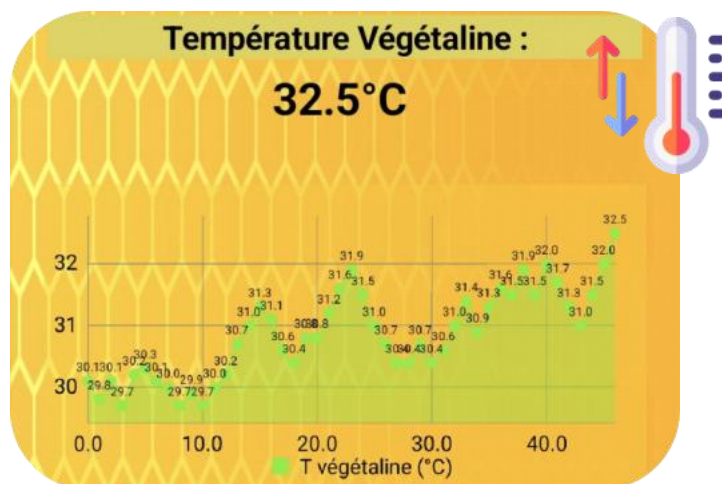
7.2 Traitement des données Bluetooth

L'application reçoit des messages de la forme DATA;T_ruche°C;T_végétaline°C. Seuls les messages commençant par le préfixe DATA sont traités. Ils sont découpés par le séparateur « ; » pour former une liste de trois éléments.

La conversion chaîne → nombre flottant est effectuée par une procédure dédiée nommée conversion. Par exemple, pour la chaîne "26,5°C" : les deux premiers caractères donnent la partie entière (26), multipliée par 1 ; le quatrième caractère donne le dixième (5), multiplié par 0,1 ; la somme donne 26,5, stockée dans la variable correspondante.

7.3 Affichage et graphiques

Les deux graphiques affichent l'évolution temporelle des températures de la ruche et de la végétaline. L'axe des abscisses est incrémenté à chaque nouvelle donnée reçue (toutes les 3 secondes). Les valeurs numériques actuelles sont également affichées dans des labels au-dessus de chaque graphique.



7.4 Identité visuelle

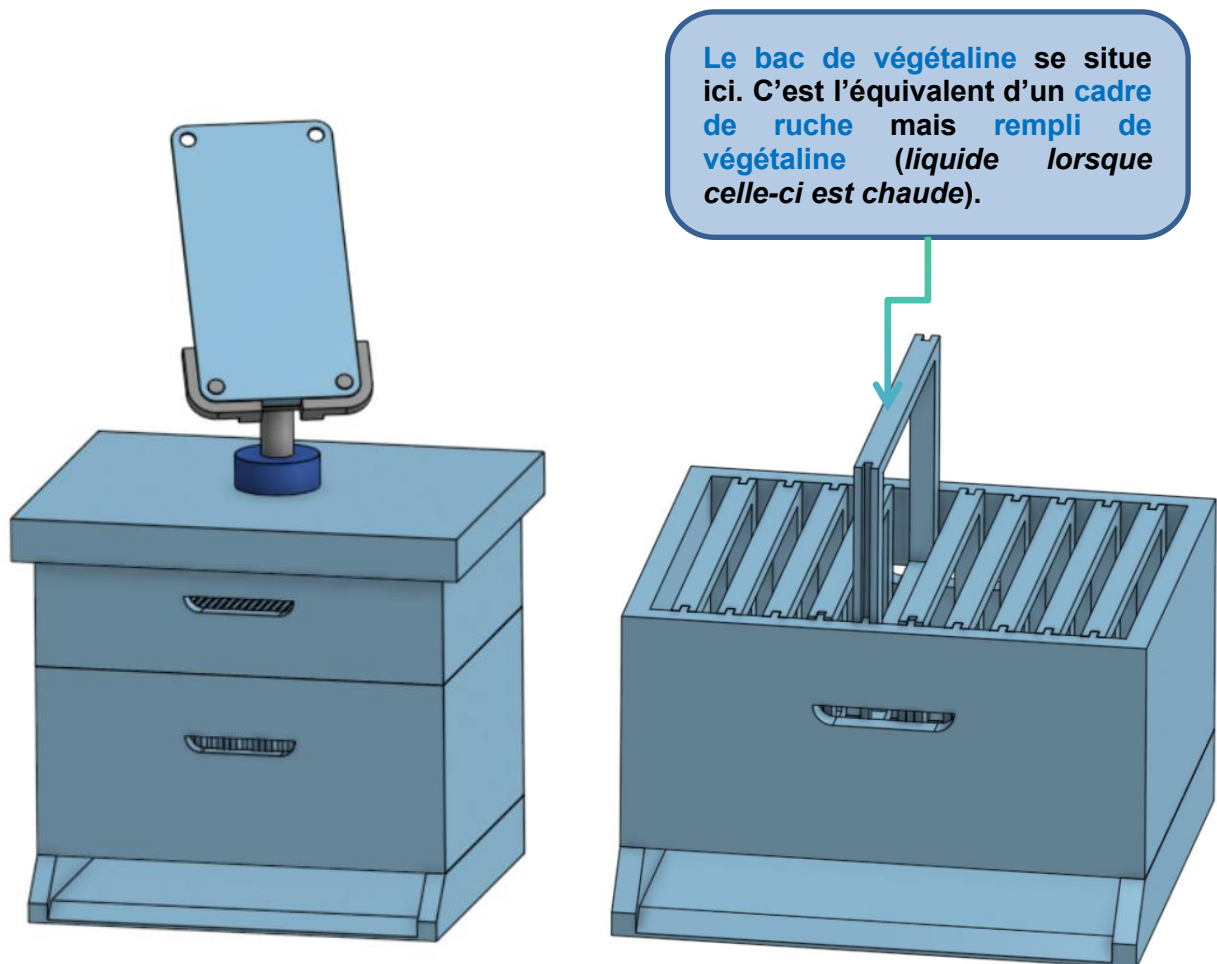
Le logo BeeWarm a été conçu sur Canva. Il s'inspire de l'univers graphique de l'apiculture : couleurs chaudes (jaune, ambre), motif en alvéoles et abeille stylisée. Le nom BeeWarm combine le mot anglais bee (abeille) et warm (chaud / chaleureux).



8. Conception mécanique (modélisation 3D et fabrication)

8.1 Ruche en 3D

Dès la première séance, une modélisation 3D de la ruche standard Langstroth (corps : 518 × 423 × 240 mm) a été réalisée sous OnShape afin de visualiser l'intégration du cadre chauffant (contenant la végétaline) et du support du panneau solaire.

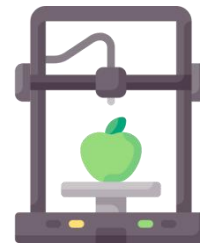


8.2 Système de suivi solaire motorisé

Le suivi automatique de la position du soleil est réalisé par un moteur pas à pas commandé par un second ESP32 externe. Deux photo-résistances, placées sous une plaque faisant de l'ombre, forment un pont diviseur de tension alimenté en 5 V. La tension aux bornes d'une des LDR est mesurée par l'ESP32 et comparée à $VCC/2$: si elle est inférieure, le moteur tourne dans un sens ; si elle est supérieure, dans l'autre sens. L'objectif est d'équilibrer l'éclairement entre les deux capteurs.

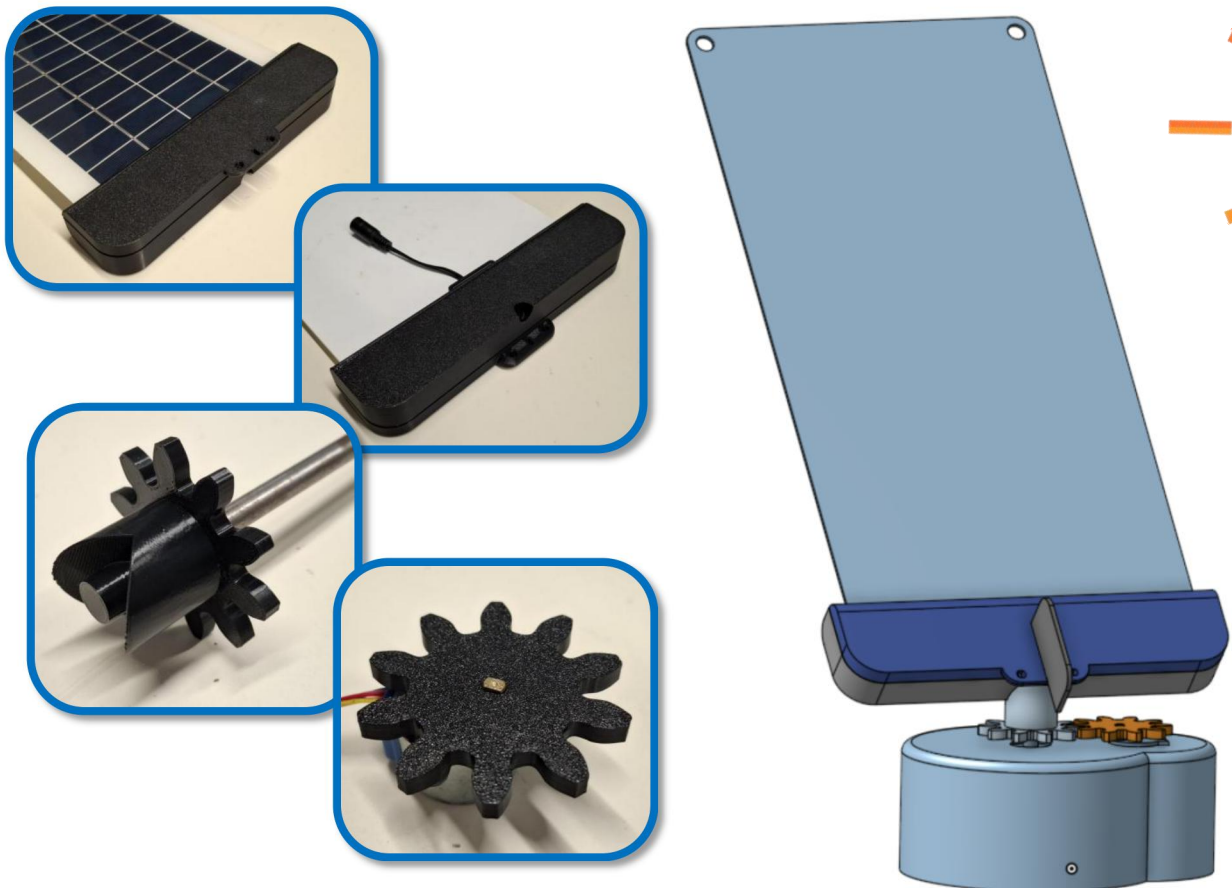
Après avoir constaté que le moteur ne pouvait pas supporter directement le poids du panneau et de son support (axe trop petit), le système a été repensé avec :

- une tige métallique centrale (longueur 89 mm, coupée à la scie à métaux et poncée) comme axe de rotation du panneau ;
- des roulements à billes pour guider l'axe et absorber les charges radiales ;
- un système d'engrenages entre le moteur (placé latéralement) et l'axe, conçu et imprimé en 3D sous OnShape ;
- un bac principal imprimé en PETG (imprimantes Prusa MK4 du FabLab) accueillant l'ensemble mécanique.



8.3 Fabrication additive

L'ensemble des pièces mécaniques (bac principal, support engrenage, support panneau solaire avec plaque d'ombre, cylindre d'assemblage) a été modélisé sous OnShape puis imprimé en PETG sur les imprimantes Prusa MK4 du FabLab de Polytech. Ce matériau a été choisi pour sa résistance à la chaleur et aux UV, adapté à une utilisation en extérieur.



9. Assemblage et tests

9.1 Assemblage mécanique

L'assemblage mécanique a été réalisé progressivement :

8. Collage de l'engrenage du panneau sur son support en 3D à la colle cyanoacrylate ;
9. Installation et câblage des photo-résistances avec gaines thermorétractables ;
10. Coupe et ponçage de l'axe métallique (89 mm) ;
11. Montage des roulements à billes dans leur logement ;
12. Assemblage des engrenages entre moteur et axe ;
13. Intégration du tout dans le bac principal imprimé en 3D.



9.2 Tests électroniques

Lors de la première mise sous tension de la PCB assemblée, aucun courant n'était tiré par la carte (problème d'alimentation de la Powerline). Après analyse du schéma sur ordinateur, le défaut a été localisé et corrigé par soudure directe d'un câble sur la PCB. Une fois ce correctif appliqué, la carte a fonctionné normalement.

Les tests ont ensuite porté sur :

- vérification de la communication Bluetooth avec le smartphone (détection de BeeWarmESP32 dans l'application) ;
- vérification de la lecture des capteurs DS18B20 (nombre de capteurs détectés, affichage des adresses sur le port série) ;
- vérification de la mesure INA219 (tensions et courant affichés sur le port série) ;
- test de la commande PWM des résistances chauffantes (oscilloscope) ;
- test de l'envoi de consigne depuis l'application (commande TEMP=XX).

Problèmes rencontrés et solutions apportées

- ESP32 non détecté (USB) → installation des pilotes FTDI232 et mise à jour Arduino IDE 2.x
- Bluetooth non démarrable → alimentation 12 V requise (prise défectueuse → câbles soudés)
- PCB sans courant → Powerline non alimentée → câble correctif soudé directement
- Axe moteur trop fragile → reprise mécanique avec engrenages + roulements à billes
- ~60 erreurs DRC PCB → correction manuelle, routage révisé, validation Eagle

10. Résultats et perspectives

10.1 État du système en fin de projet

À l'issue des séances de projet, le système BeeWarm est opérationnel dans ses fonctions principales :

- La PCB est assemblée, alimentée et fonctionnelle ;
- Les capteurs de température DS18B20 remontent correctement les données ;
- L'INA219 mesure en continu la puissance disponible sur le bus solaire ;
- Le Bluetooth fonctionne et permet l'affichage en temps réel des températures dans l'application ;
- La modification de la consigne depuis l'application est opérationnelle ;
- L'algorithme MPPT P&O est implémenté et s'exécute à chaque cycle.

10.2 Limites actuelles

- Le système de suivi solaire motorisé n'a pas encore été entièrement intégré avec la PCB principale en conditions réelles.
- L'algorithme MPPT P&O peut présenter des oscillations autour du MPP par temps instable ; un algorithme Incremental Conductance pourrait améliorer la précision.
- L'application mobile n'affiche pas encore les données de puissance/énergie solaire (seules les températures sont transmises via Bluetooth dans la version actuelle).
- La résistance chauffante et la végétaline n'ont pas été testées en conditions hivernales réelles sur la ruche.

10.3 Améliorations futures suggérées

- Intégrer la transmission des données INA219 (V, I, P) dans le message Bluetooth et les afficher dans l'application ;
- Ajouter un module de stockage d'énergie (batterie LiPo + circuit de charge MPPT dédié) pour un fonctionnement nocturne ;
- Implémenter un algorithme MPPT plus robuste (Incremental Conductance) ;
- Finaliser l'intégration du suivi solaire motorisé et le tester sur la ruche ;
- Passer à une communication Wi-Fi pour un monitoring à distance (tableau de bord web) ;
- Réaliser des mesures de performance énergétique réelles sur une saison complète.

11. Annexes et Bibliographie

11.1 Bibliothèques Arduino utilisées

Bibliothèque	Version	Usage
BluetoothSerial.h	Intégrée ESP32	Communication Bluetooth Serial
OneWire.h	–	Bus 1-Wire pour DS18B20
DallasTemperature.h	–	Interface capteurs DS18B20
Wire.h	Intégrée Arduino	Bus I ² C (INA219)
Adafruit_INA219.h	–	Driver capteur INA219

11.2 Sites et ressources de référence

- Documentation ESP32 Arduino : <https://docs.espressif.com/projects/arduino-esp32>
- Adafruit INA219 library : https://github.com/adafruit/Adafruit_INA219
- MIT App Inventor : <https://appinventor.mit.edu>
- OnShape (modélisation 3D) : <https://www.onshape.com>
- Eagle / Fusion 360 Electronics : <https://www.autodesk.com/products/eagle>
- Prusa Research (imprimantes 3D) : <https://www.prusa3d.com>

11.3 Code source

Le code source Arduino complet est fourni en annexe du présent mémoire. Deux versions du programme sont disponibles :

- App_BT_Temp_BeeWarm_avec_set.ino : version intermédiaire utilisée durant le développement du projet ;
- BeeWarm_final.ino : version finale du programme Arduino utilisée pour le prototype définitif.

L'application mobile est également fournie sous deux formats :

- BeeWarm.apk : fichier d'installation permettant d'installer directement l'application sur un smartphone Android ;
- BeeWarm.aia : projet App Inventor permettant de modifier ou faire évoluer l'application si nécessaire.

L'ensemble des fichiers du projet est également versionné sur le dépôt GitHub du projet.